FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

SISTEMAS DISTRIBUIDOS

Profesor: John Corredor, Ph.D.

GESTIÓN INTELIGENTE DE TRÁFICO URBANO

Primera Entrega

Presentado por:

Juan Diego Ariza

Nicolas Leon

Juan Diego Pardo

Juan Sebastian Urbano

Bogotá D.C., Colombia
Abril de 2026

Índice

1. Introducción **2**

2. Modelos del Sistema **2**
2.1. Modelo Arquitectónico . . . 2
2.2. Modelo de Interacción . . . 3
2.3. Modelo de Fallos . . . 3
2.4. Modelo de Seguridad . . . 4

3. Diseño del Sistema **5**
3.1. Diagrama de Despliegue . . . 5
3.2. Diagrama de componentes . . . 6
3.3. Diagrama de Clases . . . 7
3.4. Diagrama de Secuencia — Flujo Principal . . . 8
3.5. Diagrama de Secuencia — Tolerancia a Fallas . . . 9

4. Parámetros del Sistema **10**
4.1. Configuración Inicial de los Procesos . . . 10
4.2. Reglas, Consultas e Indicaciones Directas . . . 11
4.2.1. Reglas de tráfico . . . 11
4.2.2. Tipos de consulta que harán los usuarios . . . 12
4.2.3. Ejemplos de indicaciones directas . . . 12
4.2.4. Tiempos de semáforo por estado . . . 13

5. Protocolo de Pruebas **14**
5.1. Pruebas Funcionales . . . 14
5.2. Pruebas de Integración . . . 15
5.3. Pruebas de Tolerancia a Fallas . . . 16
5.4. Pruebas de Desempeño . . . 16

6. Estrategias para Obtener Métricas de Desempeño **17**
6.1. Métrica 1 — Solicitudes almacenadas en la BD en 2 minutos . . . 17
6.2. Métrica 2 — Tiempo desde indicación directa hasta cambio de semáforo . . . 18
6.3. Resumen de herramientas de medición . . . 19

7. Implementación Inicial — PC1, PC2 y PC3 **19**
7.1. Estructura del Proyecto . . . 19
7.2. Servicios PC1 — Sensores y Broker . . . 20
7.3. Servicios PC2 — Analítica y Control . . . 20
7.4. Servicios PC3 — BD Principal y Monitoreo . . . 21
7.5. Verificación del Sistema . . . 21

1. Introducción

El presente informe corresponde a la primera entrega del proyecto de la asignatur Introducción a los Sistemas Distribuidos, cuyo objetivo es diseñar e implementar una plataforma distribuida para la gestión inteligente del tráfico urbano. El sistema monitorea, analiza y reacciona ante condiciones de tráfico en tiempo real utilizando múltiples componentes distribuidos que se comunican mediante la biblioteca ZeroMQ.

La ciudad simulada se representa como una matriz de 5×6 intersecciones (filas A–E, columnas 1–6), para un total de 30 intersecciones controladas por semáforos inteligentes. En cada intersección operan tres tipos de sensores: espiras inductivas, cámaras de tráfico y sensores GPS, que generan eventos periódicos relacionados con el volumen vehicular, la velocidad promedio y el nivel de ocupación de las vías. Esto da un total de 90 sensores simulados en el sistema.

El sistema se distribuye en tres máquinas virtuales con Ubuntu Linux, con responsabilidades claramente diferenciadas: PC1 aloja los sensores, el broker ZeroMQ y los semáforos; PC2 ejecuta el servicio de analítica, el control de semáforos, la base de datos réplica y el componente de health check; y PC3 aloja el servicio de monitoreo y consulta junto con la base de datos principal.

Este informe presenta el diseño completo del sistema, incluyendo los modelos fundamentales de sistemas distribuidos aplicados al proyecto, los diagramas de despliegue, componentes, clases y secuencia, los parámetros de configuración, las reglas de tráfico definidas, el protocolo de pruebas con énfasis en desempeño, las estrategias para obtener las métricas requeridas, y la implementación inicial de los servicios de PC1, PC2 y PC3.

2. Modelos del Sistema

2.1. Modelo Arquitectónico

El sistema sigue una arquitectura distribuida en tres capas físicas correspondientes a PC1, PC2 y PC3. La comunicación entre capas se realiza exclusivamente mediante ZeroMQ, lo que garantiza desacoplamiento entre productores y consumidores de datos.

La arquitectura combina dos estilos complementarios:

Publicación/Suscripción (PC1 $\rightarrow$ PC2): los sensores publican eventos sin conocer quién los consume. El broker actúa como intermediario desacoplando completamente la capa de generación de datos de la capa de procesamiento. Esto permite agregar o quitar sensores sin modificar el resto del sistema.

Cliente/Servidor (PC3 $\leftrightarrow$ PC2): el servicio de monitoreo actúa como cliente que envía consultas e indicaciones directas al servicio de analítica mediante REQ/REP. Esta comunicación es síncrona porque el usuario necesita una respuesta inmediata a sus consultas.

La distribución de responsabilidades es la siguiente:

- **PC1** genera datos y actúa como capa de percepción del sistema. Aloja los 90 sensores simulados, el broker ZeroMQ y los 30 semáforos.
- **PC2** procesa, decide y controla. Es el núcleo del sistema. Aloja el servicio de analítica, el control de semáforos, la base de datos réplica y el health check.
- **PC3** persiste datos y permite la interacción con el usuario. Aloja la base de datos principal y el servicio de monitoreo y consulta.

2.2. Modelo de Interacción

El sistema utiliza dos tipos de comunicación según el contexto y los requisitos de cada componente:

Comunicación asíncrona (PUB/SUB y PUSH/PULL): utilizada para el flujo de datos de sensores y comandos de control. Los procesos no se bloquean esperando respuesta, lo que permite que el sistema procese múltiples eventos simultáneamente. Si un componente falla temporalmente, los mensajes se encolan en los buffers internos de ZeroMQ sin pérdida de datos.

Comunicación síncrona (REQ/REP): utilizada únicamente para consultas del usuario desde PC3 hacia el servicio de analítica en PC2. Se elige este patrón porque el usuario necesita una respuesta confirmada antes de continuar.

Los tiempos de interacción relevantes del sistema son los siguientes:

Cuadro 1: Tiempos de interacción del sistema

Parámetro	**Valor**
Intervalo de generación de eventos por sensor	10 segundos
Tiempo de semáforo en condición normal	15 s verde / 15 s rojo
Tiempo de semáforo en congestión	30 s verde / 10 s rojo
Tiempo de semáforo en priorización	45 s verde en vía priorizada
Intervalo de heartbeat del health check	5 segundos
Timeout para detectar falla de PC3	15 segundos
Intervalo de reintento de reconexión	30 segundos

2.3. Modelo de Fallos

El único fallo contemplado es la **caída de PC3**. Este fallo se clasifica como un *fallo de omisión*: PC3 deja de responder sin aviso previo, dejando de procesar mensajes entrantes y de responder solicitudes.

El mecanismo de detección y recuperación opera en tres fases:

Detección: el componente HealthCheck en PC2 envía un heartbeat cada 5 segundos al servicio de monitoreo en PC3 mediante REQ/REP. Si no recibe respuesta en 15 segundos (equivalente a 3 intentos fallidos consecutivos), declara PC3 como caído e inicia el proceso de enmascaramiento.

Enmascaramiento: una vez detectada la falla, HealthCheck notifica al servicio de analítica para que redirija todas las escrituras a la BD réplica en PC2, la cual ya estaba siendo actualizada simultáneamente durante la operación normal. La operación es completamente transparente para el usuario: el sistema continúa procesando eventos y controlando semáforos sin interrupción.

Recuperación: el HealthCheck continúa enviando heartbeats cada 30 segundos. Cuando PC3 responde nuevamente, se sincroniza la BD principal con los eventos generados durante el período de caída (delta de datos), y el sistema vuelve a operar con la BD principal en PC3 como fuente de verdad.

Cabe destacar que el sistema no contempla fallos de PC1 o PC2 según lo especificado en el enunciado. Sin embargo, ZeroMQ maneja internamente desconexiones temporales mediante reconexión automática, lo que proporciona cierta resiliencia ante interrupciones breves de red.

2.4. Modelo de Seguridad

El sistema opera en una red de confianza (red local de la universidad sobre máquinas virtuales con Ubuntu Linux) por lo que no se implementan mecanismos de autenticación ni cifrado. Sin embargo, se identifican las siguientes amenazas y las medidas básicas adoptadas para mitigarlas:

Integridad de mensajes: todos los eventos incluyen los campos `sensor_id` y `timestamp`, que permiten al servicio de analítica detectar mensajes duplicados o fuera de orden antes de procesarlos.

Disponibilidad: garantizada mediante la réplica de BD en PC2 y el mecanismo de health check descrito en el modelo de fallos. El sistema está diseñado para operar de forma ininterrumpida ante la caída de PC3.

Validación de entradas: el servicio de analítica valida que los campos requeridos estén presentes y dentro de rangos razonables en cada evento antes de procesarlo. Mensajes malformados o con valores fuera de rango son descartados con un mensaje de error registrado en pantalla y en la base de datos.

Control de acceso a indicaciones directas: el servicio de monitoreo en PC3 es el único punto autorizado para enviar indicaciones directas al servicio de analítica. En una versión productiva del sistema se agregaría autenticación mediante tokens en este canal REQ/REP para evitar comandos no autorizados.

3. Diseño del Sistema

3.1. Diagrama de Despliegue

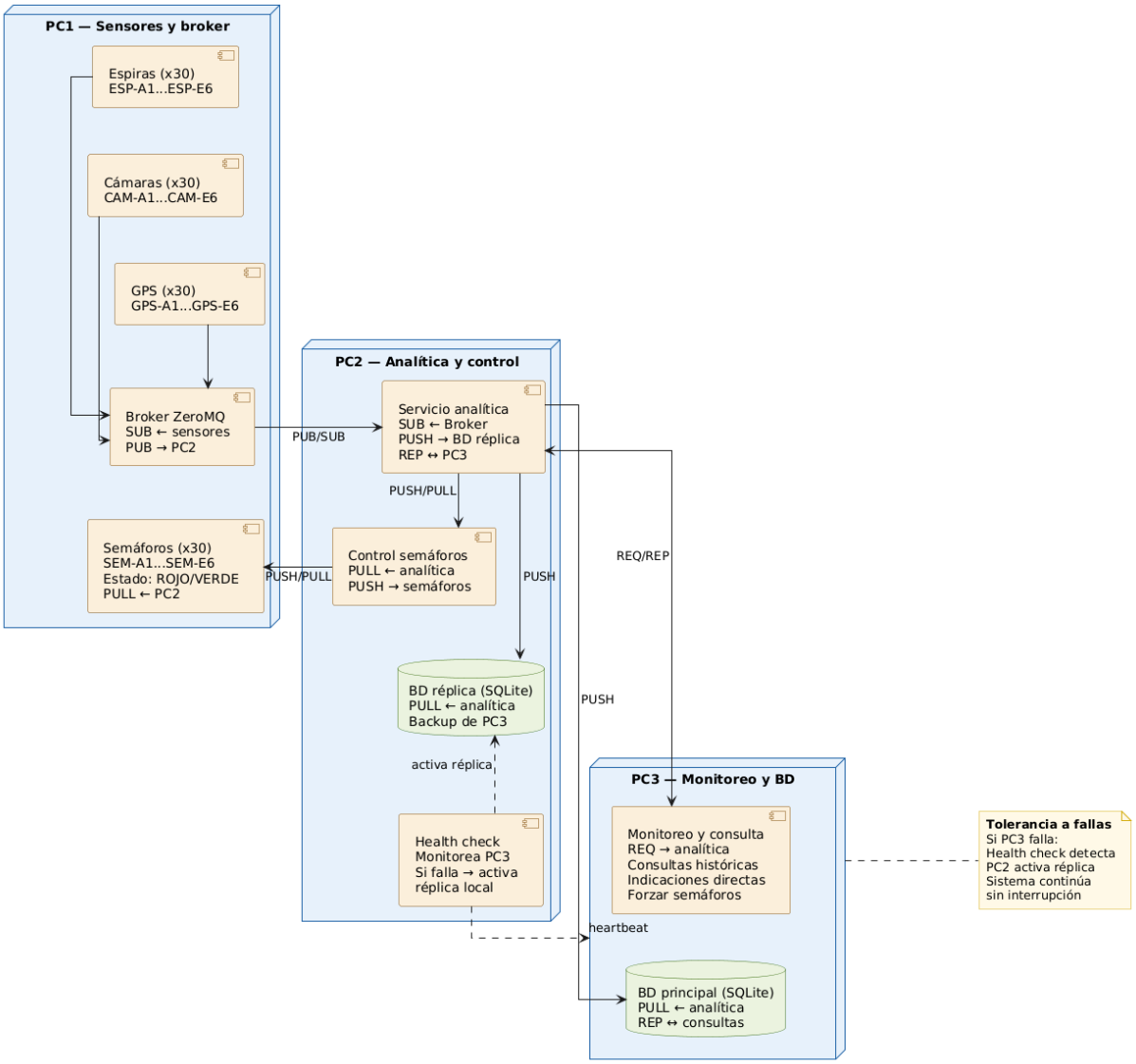

Figura 1: Diagrama de despliegue del sistema

3.2. Diagrama de componentes

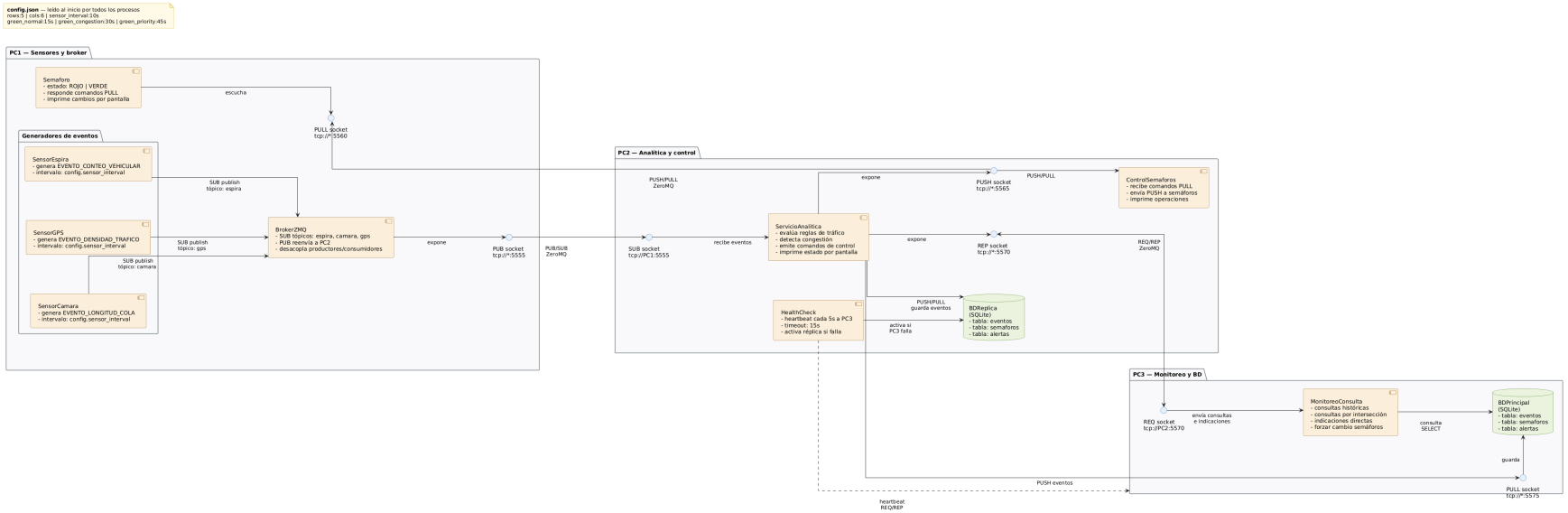

Figura 2: Diagrama de componentes del sistema

3.3. Diagrama de Clases

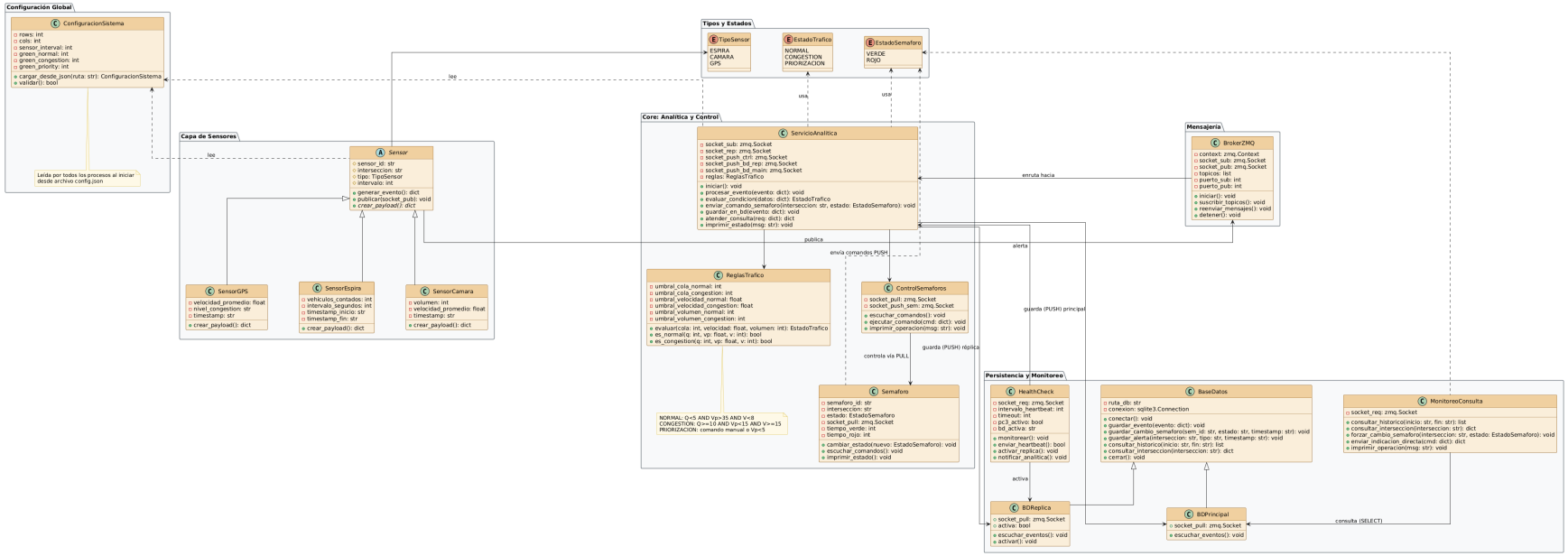

Figura 3: Diagrama de clases del sistema

3.4. Diagrama de Secuencia — Flujo Principal

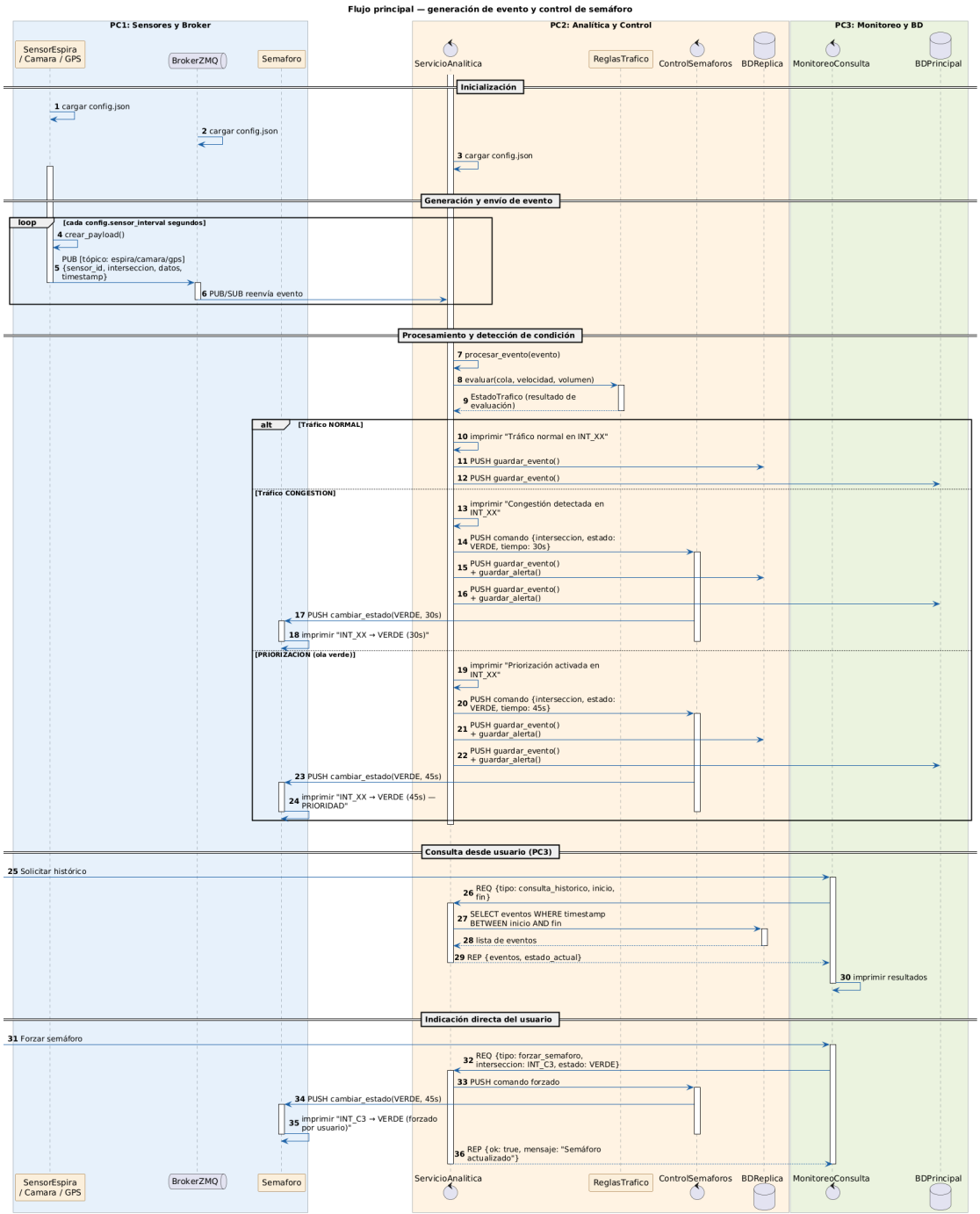

Figura 4: Diagrama de secuencia — flujo principal

3.5. Diagrama de Secuencia — Tolerancia a Fallas

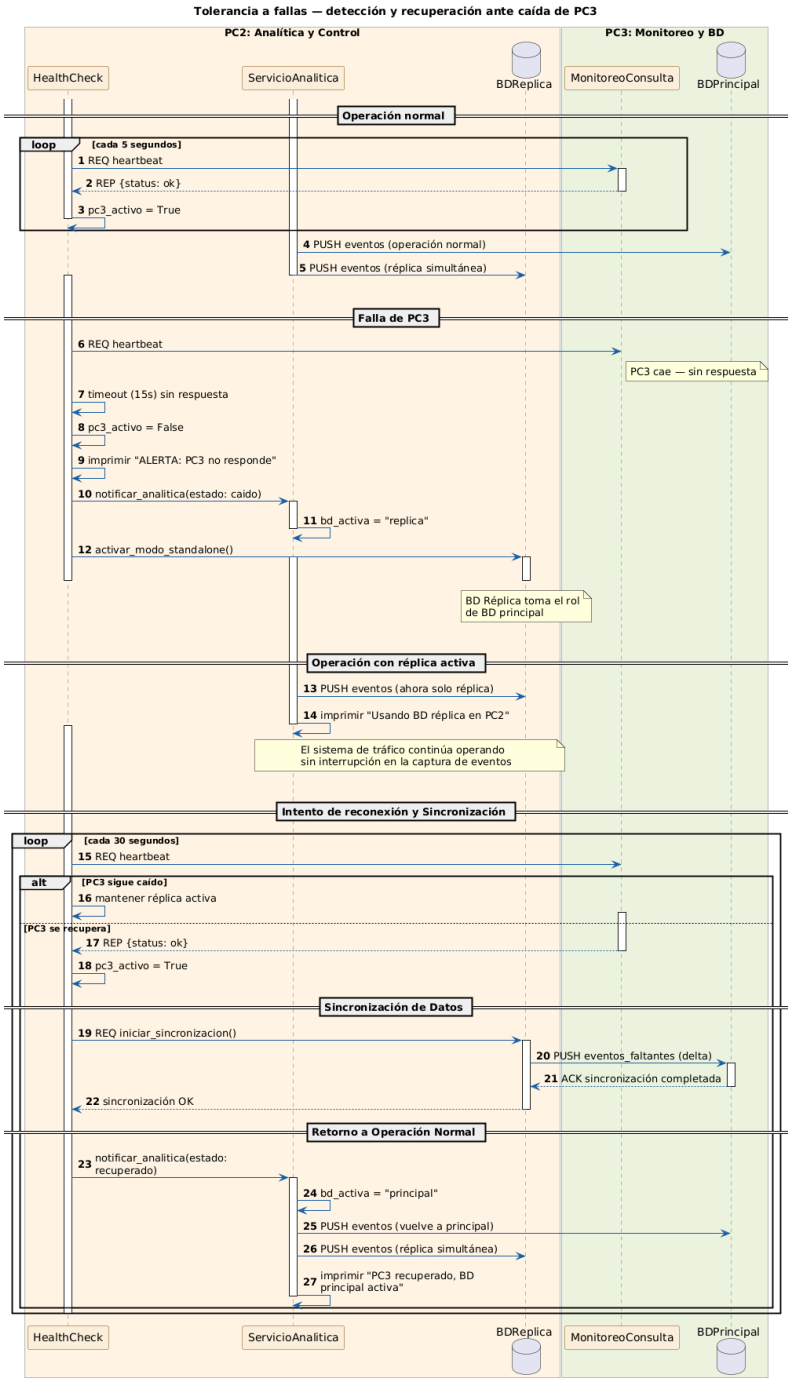

Figura 5: Diagrama de secuencia — tolerancia a fallas PC3

4. Parámetros del Sistema

4.1. Configuración Inicial de los Procesos

Todos los procesos del sistema obtienen su configuración inicial mediante la lectura de un archivo `config.json` ubicado en un directorio compartido accesible desde las tres máquinas. Este archivo es leído por cada proceso al momento de iniciar, antes de establecer cualquier conexión ZeroMQ. El archivo centraliza todos los parámetros del sistema, lo que permite modificar el comportamiento del sistema sin recompilar ni modificar el código fuente.

El archivo `config.json` tiene la siguiente estructura:

```json
{
  "ciudad": {
    "rows": 5,
    "cols": 6,
    "filas": ["A", "B", "C", "D", "E"]
  },
  "sensores": {
    "intervalo_segundos": 10,
    "tipos": ["espira", "camara", "gps"]
  },
  "semaforos": {
    "tiempo_verde_normal": 15,
    "tiempo_rojo_normal": 15,
    "tiempo_verde_congestion": 30,
    "tiempo_rojo_congestion": 10,
    "tiempo_verde_priorizacion": 45
  },
  "red": {
    "ip_pc1": "10.43.100.24",
    "ip_pc2": "10.43.100.179",
    "ip_pc3": "10.43.98.223",
    "puerto_broker_pub": 5555,
    "puerto_broker_sub": 5556,
    "puerto_analitica_rep": 5570,
    "puerto_ctrl_semaforos": 5565,
    "puerto_semaforos_pull": 5560,
    "puerto_bd_principal": 5575,
    "puerto_bd_replica": 5576,
    "puerto_healthcheck": 5580
  },
  "healthcheck": {
    "intervalo_heartbeat": 5,
    "timeout_segundos": 15,
    "intervalo_reconexion": 30
  }
}
```

Listing 1: Estructura del archivo config.json

A partir de este archivo cada proceso deriva sus parámetros específicos:

- **Sensores (PC1):** determinan el número de intersecciones a cubrir (`rows` × `cols` = 30), el identificador único de cada sensor (por ejemplo `ESP-A1`, `CAM-C5`, `GPS-E6`), y el intervalo de generación de eventos.
- **Broker ZeroMQ (PC1):** obtiene los puertos en los que debe escuchar las publicaciones de los sensores y en los que debe reenviar los eventos hacia PC2.
- **Semáforos (PC1):** determinan los tiempos de permanencia en cada estado (VERDE/ROJO) según la condición de tráfico activa, y el puerto en el que escuchan comandos del servicio de control.
- **Servicio de analítica (PC2):** obtiene las direcciones IP y puertos de todos los componentes con los que se comunica, así como los umbrales para evaluar las reglas de tráfico.
- **Control de semáforos (PC2):** obtiene el puerto de escucha de comandos y la dirección de los semáforos en PC1.
- **Health check (PC2):** obtiene el intervalo de heartbeat, el timeout para declarar falla y el intervalo de reintento de reconexión.
- **Monitoreo y consulta (PC3):** obtiene la dirección y puerto del servicio de analítica en PC2 para enviar consultas e indicaciones directas.

El número total de sensores se calcula automáticamente como rows×cols×3 = 5×6×3 = 90 sensores, y el número de semáforos como rows × cols = 30 semáforos, uno por intersección.

4.2. Reglas, Consultas e Indicaciones Directas

4.2.1. Reglas de tráfico

El servicio de analítica evalúa el estado de tráfico de cada intersección combinando los datos recibidos de los tres tipos de sensores. Las variables utilizadas son:

- Q: longitud de cola en vehículos (sensor cámara)
- Vp: velocidad promedio en km/h (sensor GPS)
- V: volumen vehicular en vehículos por minuto (sensor espira)

Las reglas definidas para cada estado son las siguientes:

Cuadro 2: Reglas de estado de tráfico por intersección

Estado	**Cola (Q)**	**Velocidad (Vp)**	**Volumen (V)**
Normal	$Q < 5$	$Vp > 35$ km/h	$V < 8$ veh/min
Congestión	$Q \geq 10$	$Vp < 15$ km/h	$V \geq 15$ veh/min
Priorización	Comando manual del usuario o $Vp < 5$ km/h (GPS)		

Cuando los valores se encuentran entre los umbrales de normal y congestión, el sistema mantiene el estado normal con los tiempos de semáforo estándar de 15 segundos. El estado de priorización puede ser activado tanto automáticamente cuando el GPS detecta velocidades menores a 5 km/h (indicativo del paso de vehículos de emergencia), como manualmente mediante una indicación directa del operador desde el servicio de monitoreo en PC3.

4.2.2. Tipos de consulta que harán los usuarios

El servicio de monitoreo y consulta en PC3 permite dos tipos de consulta mediante el patrón REQ/REP hacia el servicio de analítica en PC2:

1. **Consulta histórica por rango de tiempo:** el usuario especifica un intervalo de tiempo (por ejemplo, hora pico de 7:00 a 9:00) y el sistema retorna todos los eventos de tráfico registrados en ese período, indicando el estado de cada intersección, el número de alertas de congestión generadas y los cambios de semáforo ejecutados.

   Ejemplo de solicitud:

```
{
  "tipo": "consulta_historico",
  "inicio": "2026-04-10T07:00:00Z",
  "fin": "2026-04-10T09:00:00Z"
}
```

Listing 2: Consulta histórica

2. **Consulta puntual por intersección:** el usuario especifica una intersección y el sistema retorna el estado actual de tráfico, el estado del semáforo (VERDE/ROJO), el último evento registrado de cada tipo de sensor y el historial reciente de alertas.

   Ejemplo de solicitud:

```
{
  "tipo": "consulta_interseccion",
  "interseccion": "INT-C5"
}
```

Listing 3: Consulta puntual por intersección

4.2.3. Ejemplos de indicaciones directas

El servicio de monitoreo también puede enviar indicaciones directas al servicio de analítica para forzar cambios de estado independientemente de los datos de los sensores. Los casos de uso principales son:

1. **Paso de ambulancia — priorización de vía:** el operador fuerza el estado VERDE en todos los semáforos de una vía completa (por ejemplo, toda la columna 3) para permitir el paso sin interrupciones.

```
{
  "tipo": "forzar_priorizacion",
  "via": "columna",
  "identificador": "3",
  "estado": "VERDE",
  "duracion_segundos": 45,
  "motivo": "paso_ambulancia"
}
```

Listing 4: Indicación directa — paso de ambulancia

2. **Forzar semáforo individual:** el operador cambia el estado de un semáforo específico en una intersección puntual.

```
{
  "tipo": "forzar_semaforo",
  "interseccion": "INT-C3",
  "estado": "VERDE",
  "duracion_segundos": 45
}
```

Listing 5: Indicación directa — semáforo individual

3. **Restablecer estado normal:** el operador cancela cualquier priorización activa y devuelve todos los semáforos al ciclo normal de 15 segundos.

```
{
  "tipo": "restablecer_normal",
  "alcance": "global"
}
```

Listing 6: Indicación directa — restablecer estado normal

Todas las indicaciones directas son registradas en la base de datos con timestamp, tipo de operación y usuario que la emitió, para efectos de auditoría y consulta histórica.

4.2.4. Tiempos de semáforo por estado

Cuadro 3: Tiempos de semáforo según estado de tráfico

Estado	**Tiempo verde**	**Tiempo rojo**
Normal	15 s	15 s
Congestión	30 s	10 s
Priorización	45 s	— (resto en rojo)

5. Protocolo de Pruebas

El protocolo de pruebas cubre cuatro categorías: pruebas funcionales, de integración, de tolerancia a fallas y de desempeño.Todas las pruebas se realizan con el sistema desplegado en tres máquinas virtuales con Ubuntu Linux según la arquitectura definida.

5.1. Pruebas Funcionales

Las pruebas funcionales verifican que cada componente del sistema se comporte correctamente de forma individual.

Cuadro 4: Pruebas funcionales por componente

Componente	**Prueba**	**Resultado esperado**
Sensor espira	Ejecutar proceso y verificar salida por pantalla	Genera evento JSON con campos: sensor_id, vehiculos_contados, intervalo_segundos, timestamp
Sensor cámara	Ejecutar proceso y verificar salida por pantalla	Genera evento JSON con campos: sensor_id, volumen, velocidad_promedio, timestamp
Sensor GPS	Ejecutar proceso y verificar salida por pantalla	Genera evento JSON con campos: sensor_id, velocidad_promedio, nivel_congestion, timestamp
Broker ZeroMQ	Verificar que reenvía eventos de los tres tópicos	Imprime por pantalla cada evento recibido y reenviado
Servicio analítica	Enviar evento con valores de congestión y verificar salida	Imprime detección de congestión y emite comando de control
Control semáforos	Enviar comando PUSH y verificar cambio	Semáforo cambia de estado e imprime operación
Monitoreo y consulta	Ejecutar consulta histórica y puntual	Retorna datos correctos desde la BD
Health check	Verificar heartbeat con PC3 activo	Imprime confirmación de conexión cada 5 segundos

5.2. Pruebas de Integración

Las pruebas de integración verifican la comunicación correcta entre los componentes de diferentes PCs.

Cuadro 5: Pruebas de integración entre componentes

Integración	**Prueba**	**Resultado esperado**
PC1 → PC2 (PUB/-SUB)	Activar sensores y verificar recepción en analítica	Analítica recibe y procesa todos los eventos generados
PC2 → PC1 (PUSH/-PULL)	Detectar congestión y verificar cambio en semáforo	Semáforo cambia a VERDE en menos de 1 segundo
PC3 → PC2 (REQ/-REP)	Enviar consulta histórica desde monitoreo	Analítica responde con datos correctos de la BD
PC2 → PC3 (PUSH)	Verificar que eventos llegan a BD principal	BD principal en PC3 contiene los mismos registros que BD réplica en PC2
PC3 → PC2 indicación directa	Forzar semáforo desde monitoreo y verificar cambio	Semáforo cambia en menos de 2 segundos y operación queda registrada en BD

5.3. Pruebas de Tolerancia a Fallas

Cuadro 6: Pruebas de tolerancia a fallas

Escenario	**Procedimiento**	**Resultado esperado**
Caída de PC3	Con el sistema en operación normal, apagar PC3 y esperar 15 segundos	Health check detecta la falla, analítica redirige escrituras a BD réplica, sistema continúa sin interrupción
Recuperación de PC3	Volver a encender PC3 después de la caída	Health check detecta recuperación, BD principal se sincroniza con delta de datos, sistema vuelve a operar con BD principal
Consistencia de datos	Comparar BD principal y BD réplica antes y después de la caída	Ambas BDs contienen los mismos registros al final del proceso de sincronización

5.4. Pruebas de Desempeño

Las pruebas de desempeño comparan el diseño original (broker de un solo hilo) contra el diseño multihilos (broker con hilos concurrentes), según la Tabla 7. Se miden dos variables dependientes en cada escenario:

- **Cantidad de solicitudes almacenadas en la BD** en un intervalo fijo de 2 minutos.
- **Tiempo desde que el usuario envía una indicación directa hasta que el semáforo cambia de estado** (latencia extremo a extremo).

Cuadro 7: Escenarios de prueba de desempeño

Escenario (variable independiente)	**Diseño original**	**Diseño multihilos**
1 sensor de cada tipo generando datos cada 10s	Solicitudes en BD / 2 min Latencia usuario→semáforo	Solicitudes en BD / 2 min Latencia usuario→semáforo
2 sensores de cada tipo generando datos cada 5s	Solicitudes en BD / 2 min Latencia usuario→semáforo	Solicitudes en BD / 2 min Latencia usuario→semáforo

Cada escenario se ejecutará **10 veces** para obtener un promedio estadísticamente representativo. Se calcularán la media, desviación estándar y coeficiente de variación para cada variable dependiente, siguiendo la misma metodología aplicada en el taller de evaluación de rendimiento MPI.

6. Estrategias para Obtener Métricas de Desempeño

6.1. Métrica 1 — Solicitudes almacenadas en la BD en 2 minutos

Esta métrica cuantifica el throughput del sistema: cuántos eventos logra procesar y persistir el sistema en un intervalo de tiempo fijo.

Procedimiento:

1. Iniciar el sistema completo con la configuración del escenario a evaluar (número de sensores e intervalo definidos en la Tabla 7).
2. Registrar el timestamp de inicio (t_0) justo antes de activar los sensores.
3. Dejar el sistema corriendo exactamente 2 minutos (120 segundos).
4. Al finalizar, ejecutar la siguiente consulta SQL sobre la BD:

```sql
SELECT COUNT(*) as total_eventos
FROM eventos
WHERE timestamp >= :t0
  AND timestamp <= :t0 + INTERVAL '2 minutes';
```

Listing 7: Consulta para contar solicitudes en 2 minutos

5. Registrar el valor de `total_eventos` como resultado de la métrica.
6. Repetir el procedimiento 10 veces y calcular media y desviación estándar.

Herramienta: consulta directa sobre el archivo SQLite usando el cliente de línea de comandos `sqlite3` o un script Python con `sqlite3`.

6.2. Métrica 2 — Tiempo desde indicación directa hasta cambio de semáforo

Esta métrica cuantifica la latencia extremo a extremo del sistema: el tiempo que transcurre desde que el operador envía un comando desde PC3 hasta que el semáforo en PC1 efectivamente cambia de estado.

Procedimiento:

1. Con el sistema en operación normal, registrar el timestamp t_1 justo antes de enviar la indicación directa desde el servicio de monitoreo en PC3.
2. Enviar el comando de forzar semáforo a una intersección específica (por ejemplo `INT-C3`).
3. El servicio de semáforo en PC1 imprime por pantalla el cambio de estado con su timestamp. Registrar ese valor como t_2.
4. Calcular la latencia como $\Delta t = t_2 - t_1$.
5. Repetir la medición 10 veces y calcular media y desviación estándar.

Implementación: se agrega en el código del servicio de monitoreo y del semáforo el registro de timestamps con precisión de milisegundos usando `time.time()` en Python. Los valores se exportan a un archivo CSV al finalizar cada prueba para su posterior análisis.

```python
import time
import csv

# En MonitoreoConsulta (PC3)     antes de enviar el comando
t1 = time.time()
socket_req.send_json(comando)
respuesta = socket_req.recv_json()
print(f"[MONITOR] Comando enviado en t1={t1:.6f}")

# En Semaforo (PC1)     al recibir y ejecutar el comando
t2 = time.time()
print(f"[SEMAFORO] INT-C3 cambio a VERDE en t2={t2:.6f}")

# Calcular latencia (se hace en post-procesamiento)
latencia_ms = (t2 - t1) * 1000
```

Listing 8: Registro de timestamps para medir latencia

6.3. Resumen de herramientas de medición

Cuadro 8: Herramientas y metodología para obtención de métricas

Métrica	**Herramienta**	**Metodología**
Solicitudes en BD / 2 min	`sqlite3` + Python	Consulta SQL COUNT(*) sobre tabla eventos con filtro de timestamp
Latencia usuario→semáforo	`time.time()` Python	Diferencia de timestamps registrados en PC3 (envío) y PC1 (ejecución)
Análisis estadístico	Python + pandas	Media, desviación estándar y coeficiente de variación sobre 10 repeticiones

7. Implementación Inicial — PC1, PC2 y PC3

7.1. Estructura del Proyecto

El proyecto se organiza en la siguiente estructura de directorios, replicada en las tres máquinas virtuales:

```
trafico_urbano/
          config.json
          pc1/
                sensor_espira.py
                sensor_camara.py
                sensor_gps.py
                broker_zmq.py
                semaforo.py
          pc2/
                servicio_analitica.py
                control_semaforos.py
                bd_replica.py
                health_check.py
          pc3/
              bd_principal.py
              monitoreo_consulta.py
```

Listing 9: Estructura de directorios del proyecto

Todos los procesos leen el archivo `config.json` al iniciar, el cual define las IPs, puertos, parámetros de la ciudad y tiempos de semáforo. La distribución de máquinas virtuales es la siguiente:

Cuadro 9: Distribución de máquinas virtuales

Rol	Hostname	IP	Responsable
PC1	MIG220	10.43.99.161	Nicolas Leon
PC2	MIG322	10.43.100.24	Juan Diego Pardo
PC3	MIG29	10.43.98.223	Juan Sebastian Urbano

7.2. Servicios PC1 — Sensores y Broker

PC1 aloja cinco procesos que se ejecutan en paralelo en terminales independientes. Los tres sensores (`sensor_espira.py`, `sensor_camara.py`, `sensor_gps.py`) generan eventos periódicos cada 10 segundos para las 30 intersecciones de la ciudad (matriz 5×6), produciendo un total de 90 flujos de datos simultáneos. Cada sensor publica sus eventos al broker mediante el patrón PUB/SUB de ZeroMQ usando el tópico correspondiente (`espira`, `camara`, `gps`).

El broker (`broker_zmq.py`) se suscribe a los tres tópicos, recibe todos los eventos y los reenvía al servicio de analítica en PC2 mediante un socket PUB en el puerto 5555. El proceso de semáforos (`semaforo.py`) inicializa los 30 semáforos en estado ROJO y escucha comandos de control desde PC2 mediante el patrón PULL en el puerto 5560, cambiando el estado de cada semáforo según las órdenes recibidas e imprimiendo cada operación por pantalla con su timestamp.

Durante las pruebas realizadas se observó el correcto funcionamiento de todos los componentes de PC1, con los sensores generando eventos para todas las intersecciones (INT-A1 hasta INT-E6) y el broker reenviando los eventos hacia PC2 de forma continua.

7.3. Servicios PC2 — Analítica y Control

PC2 ejecuta cuatro procesos en paralelo. El servicio de analítica (`servicio_analitica.py`) es el componente central del sistema: se suscribe al broker de PC1 mediante PUB/SUB, evalúa las reglas de tráfico para cada intersección combinando los datos de los tres tipos de sensores, y toma decisiones de control. Cuando detecta congestión activa el semáforo en VERDE por 30 segundos; cuando detecta priorización lo activa por 45 segundos. Simultáneamente persiste todos los eventos en la BD réplica local y los reenvía a la BD principal en PC3 mediante PUSH/PULL. El servicio también atiende consultas e indicaciones directas desde PC3 mediante el patrón REQ/REP en el puerto 5570, usando hilos separados para no bloquear el procesamiento de eventos.

El servicio de control de semáforos (`control_semaforos.py`) recibe los comandos de la analítica mediante PULL y los reenvía a los semáforos en PC1 mediante PUSH, imprimiendo cada operación ejecutada. La base de datos réplica (`bd_replica.py`) recibe eventos mediante PULL en el puerto 5576 y los persiste en el archivo `bd_replica.db` con SQLite. El health check (`health_check.py`) envía heartbeats cada 5 segundos al servicio de BD

principal en PC3; si no recibe respuesta en 15 segundos activa automáticamente la réplica local como fuente principal de datos.

Durante las pruebas se verificó que la analítica procesa correctamente los eventos de las 30 intersecciones, detecta los estados de tráfico según las reglas definidas y emite los comandos de control correspondientes hacia los semáforos.

7.4. Servicios PC3 — BD Principal y Monitoreo

PC3 ejecuta dos procesos. La base de datos principal (`bd_principal.py`) opera con dos hilos: uno que escucha eventos desde la analítica de PC2 mediante PULL en el puerto 5575 y los persiste en `bd_principal.db`, y otro que responde los heartbeats del health check de PC2 mediante REQ/REP en el puerto 5580. El servicio de monitoreo y consulta (`monitoreo_consulta.py`) provee una interfaz interactiva de línea de comandos que permite al usuario realizar consultas históricas por rango de tiempo, consultas puntuales por intersección, forzar cambios de semáforo individuales, activar la ola verde en una vía completa y restablecer el estado normal global, todo mediante el patrón REQ/REP hacia el servicio de analítica en PC2.

7.5. Verificación del Sistema

Se verificó el funcionamiento completo del sistema ejecutando todos los procesos en las tres máquinas virtuales simultáneamente. La siguiente consulta confirmó que la BD principal en PC3 estaba recibiendo y persistiendo los eventos correctamente:

```python
import sqlite3
conn = sqlite3.connect('bd_principal.db')
c = conn.cursor()
c.execute('SELECT COUNT(*) FROM eventos')
print('Eventos en BD principal:', c.fetchone()[0])
conn.close()
# Resultado: Eventos en BD principal: 3030
```

Listing 10: Verificación de eventos en BD principal

El resultado de 3030 eventos almacenados confirmó el flujo completo de datos desde los sensores en PC1 hasta la base de datos principal en PC3, pasando por el procesamiento en PC2, operando de forma distribuida en tres máquinas virtuales con Ubuntu Linux.

Referencias

[1] Hintjens, P. (2013). *ZeroMQ: Messaging for Many Applications*. O'Reilly Media.

[2] SQLite Consortium. (2024). *SQLite Documentation*. Recuperado de https://www.sqlite.org/docs.html

[3] PyZMQ Developers. (2024). *PyZMQ Documentation*. Recuperado de https://pyzmq.readthedocs.io

[4] Corredor, J. (2026). *Proyecto: Gestión Inteligente de Tráfico Urbano*. Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas.